Weebly Theme Conversion Plan

yogavanessa.com → Weebly Custom Theme

Source repo: https://github.com/Ethan-Arrowood/yogavanessa.com
Goal: Convert the existing static HTML/CSS design into a valid, importable Weebly theme zip package. The developer (Ethan) owns site structure; the client owns content (blog posts, service card images, copy).

Context & Constraints

- The design system is already finalized — do not alter colors, fonts, or layout structure
- Color palette: `#70d9e1` (cyan), `#5848b7` (purple), `#333333` (dark text), `#666666` (medium text), `#fafafa` (light bg), `#ffffff` (white)
- Fonts: Quattrocento (headings/body), Lora (taglines only) — both loaded from Google Fonts
- The client should be able to: write blog posts, swap service card images, update copy
- The client should NOT be able to: change layout structure, move sections, alter hero/gift cert backgrounds
- Hero (`aspen-trees.jpg`) and gift certificate section (`waterfall-background.jpg`) backgrounds stay locked in the theme shell as CSS — not editable by client
- Service card images (massage, yoga, retreats, resources) should be converted to editable `<img>` elements so the client can swap them via Weebly's editor

---

Deliverables

1. `index.html` — Weebly theme shell with variables
2. `main_style.css` — all styles extracted from the inline `<style>` block
3. `theme.js` — mobile nav script extracted from inline `<script>` block

4. `manifest.json` — theme configuration
5. All image assets (flat, no subdirectories)
6. Final `yogavanessa-theme.zip` — ready to import into Weebly

Phase 1: File Structure Setup

Create the following flat file structure (Weebly requires all files in root, no subdirectories):

```
yogavanessa-theme/
├── index.html
├── main_style.css
├── theme.js
├── manifest.json
├── logo.png
├── aspen-trees.jpg
├── waterfall-background.jpg
├── massage-background.jpg
├── yoga-pose.png
├── retreats.png
├── cookbook.jpg
├── headshot.jpg
├── waterfall-video.jpg
```

Download all assets from the GitHub repo. The `old-massage_files/` directory, `old-massage.html`, and `old-yogavanessa.com.html` are legacy files — do not include them in the theme zip.

Phase 2: Create `main_style.css`

Extract the entire contents of the `<style>` block from `index.html` into `main_style.css`.

Required changes during extraction:

1. Image asset references — update all `url('filename.jpg')` references to use bare filenames (Weebly resolves these from the theme root at runtime). The existing references in the source are already bare filenames so no change is needed, but verify each one:
   - `url('aspen-trees.jpg')` ✓

- `url('waterfall-background.jpg')` ✓
- `url('massage-background.jpg')` ✓
- `url('yoga-pose.png')` ✓
- `url('retreats.png')` ✓
- `url('cookbook.jpg')` ✓

2. Remove the `.design-notes` CSS block entirely — that was a developer reference section, not part of the final design.
3. Keep all CSS exactly as-is otherwise. Do not refactor, reorder, or optimize — fidelity to the original design is the priority.

Phase 3: Create `theme.js`

Extract the inline `<script>` block at the bottom of `index.html` into `theme.js`.

The script handles mobile nav toggle. Extract as-is:

```
const hamburger = document.querySelector('.hamburger');
const navMenu = document.querySelector('nav ul');
const navLinks = document.querySelectorAll('nav ul a');

hamburger.addEventListener('click', () => {
  hamburger.classList.toggle('active');
  navMenu.classList.toggle('active');
});

navLinks.forEach(link => {
  link.addEventListener('click', () => {
    hamburger.classList.remove('active');
    navMenu.classList.remove('active');
  });
});

document.addEventListener('click', (e) => {
  if (!e.target.closest('nav')) {
    hamburger.classList.remove('active');
    navMenu.classList.remove('active');
  }
});
```

No changes needed to the logic.

Phase 4: Create `manifest.json`

This file configures the theme in Weebly's editor. Create it as follows:

```
{
  "version": "1.0",
  "name": "Blissful Life",
  "author": "Ethan Arrowood",
  "colors": [
    {
      "name": "Default",
      "primary": "#5848b7",
      "secondary": "#70d9e1",
      "background": "#ffffff",
      "text": "#333333"
    }
  ],
  "fonts": {
    "heading": "Quattrocento",
    "body": "Quattrocento"
  },
  "responsive": true
}
```

This locks the palette in the Weebly Design tab so the client sees the correct brand colors rather than platform defaults.

Phase 5: Build `index.html` (Theme Shell)

This is the core of the conversion. The theme shell contains the locked structural chrome (nav, hero, section scaffolding, footer) with Weebly template variables injected at the right points.

5.1 — `<head>` block

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>%%title%%</title>
  <link href="https://fonts.googleapis.com/css2?family=Quattrocento:wght@400;700&
  <link rel="stylesheet" href="main_style.css">
  %%header%%
</head>
```

Notes:

- `%%title%%` replaces the hardcoded `<title>` — Weebly injects the page title here
- `%%header%%` is required by Weebly and must appear before `</head>` — it injects platform scripts, SEO meta tags, and other head content
- Google Fonts link stays as-is

5.2 — Navigation

Replace the hardcoded `<nav>` with a version that uses `%%menu%%` for the link list but keeps all custom markup for the logo and hamburger button:

```
<body>
<nav>
  <div class="container">
    <a href="/" class="logo">
      <img src="logo.png" alt="Blissful Life">
    </a>
    <button class="hamburger" aria-label="Toggle menu">
      <span></span>
      <span></span>
      <span></span>
    </button>
    %%menu%%
  </div>
</nav>
```

Important: `%%menu%%` outputs a `<ul>` with Weebly-generated `<li><a>` items based on the site's pages. The existing CSS targets `nav ul` and `nav a`, which will still apply correctly to the Weebly-generated markup. The hamburger button JS targets `nav ul` by selector, which will also work. No CSS changes needed.

5.3 — Hero Section (LOCKED — not client-editable)

Keep exactly as-is from the source. This is baked into the theme shell:

```
<section class="hero" role="img" aria-label="Golden aspen trees reaching toward b
  <div class="hero-content">
    <h1>Blissful Life</h1>
    <p class="tagline">Why feel okay, when you can feel great?</p>
    <p class="hero-description">Therapeutic massage services designed to help you
    <a href="#booking" class="btn btn-primary">Book a Massage</a>
  </div>
</section>
```

The hero background image (`aspen-trees.jpg`) is set via CSS, so it stays locked. The client cannot move or remove this section.

5.4 — Main Content Area (CLIENT-EDITABLE)

This is where `%%content%%` goes. Weebly replaces this with the drag-and-drop content the client builds per page:

```
<div class="wsite-content">
  %%content%%
</div>
```

What this means for the services grid and bio section: The services section, gift cert section, bio section, and final CTA from the original `index.html` body should be rebuilt as Weebly page content (drag-and-drop blocks), NOT hardcoded in the theme shell. The theme shell only contains: nav, hero, and footer.

The developer (Ethan) will build out the homepage content in Weebly's editor after the theme is applied. The service card images will be Weebly image elements that the client can swap.

5.5 — Footer (LOCKED)

```
<footer>
  <div class="footer-content">
    <div class="footer-links">
      %%menu%%
    </div>
    <p>%%footer%%</p>
  </div>
</footer>

<script src="theme.js"></script>
%%footer_scripts%%
</body>
```

```
</html>
```

Notes:

- `%%menu%%` in the footer re-uses the same page list for footer nav links — the CSS will need a scoped override to handle footer link styling vs nav link styling (see Phase 6)
- `%%footer%%` outputs the Weebly copyright/footer text
- `%%footer_scripts%%` is required by Weebly and must appear before `</body>` — it injects platform JS
- The custom `theme.js` is loaded before `%%footer_scripts%%`

5.6 — Complete `index.html` assembled

Putting it all together:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>%%title%%</title>
  <link href="https://fonts.googleapis.com/css2?family=Quattrocento:wght@400;700&
  <link rel="stylesheet" href="main_style.css">
  %%header%%
</head>
<body>

<nav>
  <div class="container">
    <a href="/" class="logo">
      <img src="logo.png" alt="Blissful Life">
    </a>
    <button class="hamburger" aria-label="Toggle menu">
      <span></span>
      <span></span>
      <span></span>
    </button>
    %%menu%%
  </div>
</nav>

<section class="hero" role="img" aria-label="Golden aspen trees reaching toward b
  <div class="hero-content">
    <h1>Blissful Life</h1>
```

```
    <p class="tagline">Why feel okay, when you can feel great?</p>
    <p class="hero-description">Therapeutic massage services designed to help you
    <a href="#booking" class="btn btn-primary">Book a Massage</a>
  </div>
</section>

<div class="wsite-content">
  %%content%%
</div>

<footer>
  <div class="footer-content">
    <div class="footer-links">
      %%menu%%
    </div>
    <p>%%footer%%</p>
  </div>
</footer>

<script src="theme.js"></script>
%%footer_scripts%%
</body>
</html>
```

Phase 6: CSS Additions to `main_style.css`

A few additions are needed to handle Weebly-specific markup and the footer menu re-use:

6.1 — Footer menu scoping

The footer reuses `%%menu%%` which outputs the same `<ul>` structure. Add scoped styles so footer links look different from nav links:

```
/* Footer menu override */
footer .footer-links ul {
  list-style: none;
  display: flex;
  justify-content: center;
  gap: 2rem;
  margin-bottom: 2rem;
  padding: 0;
  flex-wrap: wrap;
}
```

```
footer .footer-links ul li {
  padding: 0;
}

footer .footer-links ul a {
  color: var(--primary-cyan);
  text-decoration: none;
}

footer .footer-links ul a:hover {
  text-decoration: underline;
}

@media (max-width: 768px) {
  footer .footer-links ul {
    flex-direction: column;
    gap: 1rem;
    align-items: center;
  }
}
```

6.2 — Weebly content wrapper

Add basic styles for the content wrapper div:

```
.wsite-content {
  min-height: 200px;
}
```

6.3 — Remove `.design-notes` styles

The `.design-notes` CSS block (yellow warning box used during development) should be removed entirely from `main_style.css`. Do not include it in the output.

Phase 7: Package the Zip

Weebly requires a flat zip with no top-level folder. The zip must be structured so all files are at the root level when extracted.

```
yogavanessa-theme.zip
├── index.html
├── main_style.css
├── theme.js
```

```
├── manifest.json
├── logo.png
├── aspen-trees.jpg
├── waterfall-background.jpg
├── massage-background.jpg
├── yoga-pose.png
├── retreats.png
├── cookbook.jpg
├── headshot.jpg
└── waterfall-video.jpg
```

Critical: Do NOT zip a folder — zip the files directly. On macOS, `cd` into the directory and run:

```
zip -r ../yogavanessa-theme.zip . -x "*.DS_Store" -x "__MACOSX/*"
```

On Linux:

```
zip -r yogavanessa-theme.zip index.html main_style.css theme.js manifest.json *.j
```

Phase 8: Import & Verify in Weebly

1. Log into the client's Weebly account (or a test account)
2. Open the site editor
3. Go to **Design → Change Theme → Import Theme**
4. Upload `yogavanessa-theme.zip`
5. Apply the theme

Verification checklist:

- Navigation renders with correct styling (white bg, shadow, logo visible)
- Hamburger menu works on mobile
- Hero section displays with `aspen-trees.jpg` background and overlay
- Hero text is readable (white on dark overlay)
- `%%content%%` area is present and editable in the Weebly editor
- Footer renders with correct dark background and cyan links

- Google Fonts load (Quattrocento and Lora)
- No CSS bleed between nav and footer menu styles
- Theme shows “Blissful Life” in the theme picker

Phase 9: Homepage Content Setup (Post-Import)

After the theme is imported and applied, the developer (Ethan) rebuilds the homepage content using Weebly’s editor. This is NOT part of the theme zip — it’s page content.

Rebuild these sections as Weebly content blocks on the homepage:

Services Grid — Use a 2-column layout with four service cards. Each card:

- Weebly Image element (allows client to swap images)
- Text heading (purple, 1.8rem)
- Paragraph text
- Button/link

Gift Certificates section — Use a Weebly section with a background image set via the editor (waterfall-background.jpg), text overlay, and button. Note: Weebly’s built-in background image sections apply their own overlays — adjust as needed to match the original design.

Bio Section — Standard content block with Weebly Image element (headshot.jpg), headings, paragraph text, and list.

Final CTA Section — Weebly section with solid cyan background (`#70d9e1`), heading, text, button.

Phase 10: Blog Setup

Weebly has a built-in blog feature. To enable:

1. In the site editor, go to **Pages → Add Page → Blog**
2. Name it (e.g., “Blog” or “News”)
3. The blog page will inherit the theme shell (nav, hero, footer)

Blog-specific styling — Add the following to `main_style.css` (can be added after initial

import via the Weebly code editor):

```
/* Blog post list */
.blog-post-title a {
  color: var(--secondary-purple);
  text-decoration: none;
  font-family: 'Quattrocento', serif;
}

.blog-post-title a:hover {
  color: var(--primary-cyan);
}

.blog-content {
  font-family: 'Quattrocento', serif;
  color: var(--text-dark);
  line-height: 1.8;
}
```

Note: Weebly's blog outputs specific class names (`blog-post-title` , `blog-content` , etc.). These styles ensure blog typography matches the site design system.

Known Limitations & Notes

Hero is page-agnostic — The hero section is in the theme shell, so it appears on every page with the same headline and background. If inner pages (Massage, Yoga, etc.) need different heroes, this would require either: (a) per-page CSS class overrides using `%%page_classes%%` , or (b) removing the hero from the shell and rebuilding it as page content on each page. For now, a single shared hero is acceptable.

`%%menu%%` used twice — Weebly outputs the same nav structure in both places. The footer CSS override in Phase 6 handles the visual differentiation.

Weebly Embed Code blocks — For any content that needs custom HTML/CSS beyond what the drag-and-drop editor supports (e.g., an embedded booking widget), use Weebly's Embed Code element. This is the escape hatch for the `#booking` button target.

Image compression — The source images are uncompressed. Before or after conversion, compress them (especially `retreats.png` and `aspen-trees.jpg`) using `imageoptim` , `squoosh` , or similar. Target under 300KB per image for reasonable page load.

Export/re-import note — Weebly's theme export restructures files differently than the import format expects. If you need to re-import after exporting, you'll need to manually

flatten the exported zip back to the correct structure before re-importing. Keep the source files in the GitHub repo as the source of truth, not the Weebly export.